

PRD — Adaptive AI Study Mentor (Class-Scoped MVP)

****Status:**** Draft v1

****Team size:**** 3

****Audience:**** All team members — read fully before starting your workstream.

1. Problem & Scope

The original concept (any student uploads any syllabus, LLM extracts structure, chatbot answers questions) is scoped down for this build to keep it achievable with no external APIs and a 3-person team.

****This version targets one specific semester's curriculum****, structured as:

- ****4–5 core subjects**** — every student takes all of them
- ****1 open elective**** — every student picks exactly one: ****ADT**** or ****CS****
- ****Honors (optional)**** — if a student opts in, they pick exactly one: ****AIML**** or ****CS Honors****

Each subject (core, elective, or honors) follows the same content hierarchy: ****Subject → Modules → Topics (→ Subtopics)****. There is no weekly breakdown in the source material — topics are ordered within their module, not tied to calendar weeks. Because the full curriculum is fixed and known ahead of time, we don't need a live document-parsing pipeline — it's structured once, by hand, into the database.

In scope (MVP)

- All core + elective + honors subjects, pre-structured into the database as Subject → Module → Topic (→ Subtopic), not uploaded/parsed at runtime
- Enrollment logic: auto-add all core subjects, plus the one elective and (if opted-in) one honors subject the student selects
- Synthetic data used to generate a sensible ****static baseline schedule**** before any real student uses the system
- Students input only: elective choice, honors choice (if any), weekly availability, and ongoing self-reported confidence per topic
- Adaptive layer that adjusts the schedule per student based on their actual pace and reported confidence
- Postgres (hosted on ****Supabase****) as the single data store, using Supabase Auth for student accounts
- Web interface (calendar/schedule view + availability form + confidence check-in)

Explicitly out of scope (deferred)

- ****No external APIs**** — no calls to Claude/OpenAI/any third-party service. Everything runs locally.
- ****No generic syllabus upload/parsing**** — that whole extraction pipeline is unnecessary since the syllabus doesn't change per student.
- ****Mentor chat**** — deferred to a future phase, to be built with a ****locally-hosted model**** (e.g. via Ollama) once core scheduling is working. Not part of this MVP.
- ****Quiz question bank**** — the teacher's unstructured notes exist but are not being processed into quiz questions for this version (too much extra work for the MVP). Mastery is tracked via ****self-reported confidence ratings**** instead of graded quizzes.

> Note on "no external APIs": this means no network calls to third-party AI/data services. Local Python libraries (scikit-learn, OR-Tools, NetworkX, etc.) and locally-run models are fine — they aren't external API calls, they just run on your own machine/server.

2. Users & Core Flow

****Student onboarding (one-time):****

1. All core subjects are auto-enrolled — no choice needed
2. Student selects their open elective: `ADT` or `CS`
3. Student flags whether they're taking Honors, and if so, picks `AIML` or `CS Honors`
4. Student enters weekly availability (days + hours/day)
5. System pulls the baseline schedule for each enrolled subject (already pre-generated from synthetic data) and merges/slots them into the student's actual available days across all their subjects at once

****Ongoing use:****

1. Student sees today's study items across all enrolled subjects on their calendar
2. Student marks a topic complete and gives a quick confidence rating (e.g. 1–5: "how well do you know this?")
3. That topic's confidence score updates
4. Remaining schedule re-flows: low-confidence topics get resurfaced/more time, high-confidence topics get pushed later or dropped from active review

3. Data Model (Postgres via Supabase) — LOCKED

This is the schema the team builds against. Granularity and deadline handling are now locked in based on team input (§6).

Subjects & enrollment

| Table | Key fields | Notes |

---|---|---

| `subjects` | id, name, category (`core` \| `elective` \| `honors`), group_name (nullable) |

Every concrete subject offering lives here: 4–5 core rows (group_name = NULL), 2 elective rows (`ADT`, `CS`, group_name = `open\_elective`), 2 honors rows (`AIML`, `CS

Honors`, group\_name = `honors`). `group_name` marks "pick exactly one from this set." |

| `student\_subjects` | student_id, subject_id | Populated at onboarding: all core rows auto-inserted, plus the one elective and (if opted-in) one honors row the student picked. |

Curriculum hierarchy (identical shape for core, elective, and honors subjects)

| Table | Key fields | Notes |

---|---|---

| `modules` | id, subject_id (FK), name, order_index, ****assessment_phase****

(`UT1` \| `UT2`) | Which unit test this module's content falls under — end-sem always covers everything, so it needs no flag |

| `topics` | id, module_id (FK), name, order_index, est_difficulty (1–5), est_hours | Fallback schedulable unit — used directly when a topic has no subtopics |

| `subtopics` | id, topic_id (FK), name, order_index, est_hours | ****Primary schedulable unit.**** Every topic that has subtopics is scheduled/rated at subtopic level. |

| `topic\_dependencies` | topic_id, depends_on_topic_id | Prerequisite ordering, defined at topic level only (subtopic-level dependencies would be unnecessary overhead) |

Student progress

| Table | Key fields | Notes |

---|---|---

| `students` | id (= Supabase `auth.users.id`), display_name, created_at | Thin profile table riding on Supabase Auth — no separate password/login logic to build |

| `availability` | student_id, day_of_week, hours_available | Editable anytime |

| `topic\_confidence` | student_id, topic_id, subtopic_id (nullable), confidence_rating (1–5), last_updated | One row per ****schedulable unit****: `subtopic\_id` set when the topic has subtopics, left NULL (rating the topic itself) when it doesn't |

| `schedule\_entries` | student_id, topic_id, subtopic_id (nullable), scheduled_date, status
(`planned` / `done` / `skipped`) | Same fallback rule as above — one entry per subtopic
where they exist, otherwise one per topic |

Baseline & optional deadlines

| Table | Key fields | Notes |

|---|---|---|

| `baseline\_schedules` | subject_id, topic_id, subtopic_id (nullable), suggested_order | The
synthetic-data-driven default sequence per subject, before personalization to a student's
availability |

| `exam\_dates` | subject_id, exam_type (`UT1` \| `UT2` \| `end\_sem`), exam_date | Real
structure per subject: UT1 covers the modules tagged `assessment\_phase = UT1`, UT2
covers the rest, end_sem is the final deadline covering everything. Scheduler uses
whichever exam date is nearest for a given topic's module as its urgency deadline. |

4. Synthetic Data Strategy

Since there's no historical student data yet, synthetic data serves two purposes:

1. ****Seeding initial difficulty/time estimates per topic**** — where you (or the instructor)
don't already have a confident number, generate a reasonable estimate from features you
do know: credit weight, topic type (concept vs. problem-solving vs. project), and position
in the course (later topics often assumed harder). This gives every topic a starting
`est\_difficulty` / `est\_hours` even before real quiz data exists.
2. ****Simulating synthetic student profiles**** — generate a batch of fake students with
randomized pace, starting ability, and availability patterns, and run them through the
scheduler and mastery model **before** real students touch the system. This is how you
validate that the baseline schedule and adaptive logic actually behave sensibly — and it's a
legitimate evaluation method to write up ("we validated the scheduler against N simulated
learners before deployment").

The ****static baseline schedule**** is just the output of running the scheduler once against
average synthetic behavior. Real students then get personalized adjustments layered on
top of that baseline as they use the system — so nobody has to "start from nothing," but
nobody is stuck with a one-size-fits-all plan either.

5. Team Split (3 people, parallel workstreams)

Split by layer so each person can work independently after a short Week 1 alignment session on the schema.

Member A — Data & Content Layer

- Finalize the Supabase/Postgres schema (tables above) and set up Row Level Security so students only see their own rows
- Digitize all core subjects (4–5), both elective options (ADT, CS), and both honors options (AIML, CS Honors) into `subjects` → modules → topics → subtopics`, including `topic_dependencies`
- ****Tag every module with `assessment\_phase` (UT1 or UT2)**** based on the teacher's actual "half syllabus" split for that subject — this needs confirming per subject, it's not always a clean 50/50 by module count
- Enter `exam\_dates` per subject: UT1, UT2, and end-sem dates
- Build the synthetic data generator: initial `est\_difficulty` / `est\_hours` per schedulable unit (subtopic where it exists, topic otherwise), plus a batch of synthetic student profiles for testing
- Own migrations/seed scripts

****Deliverable:**** a populated, queryable Supabase database (all subjects, real hierarchy, real exam dates) + synthetic test dataset, ready for B and C to build against.

Member B — Adaptive Engine

- Implement the confidence-based mastery update over ****schedulable units**** (subtopic if it exists, else the parent topic) — simplified math since input is a direct 1–5 rating, not graded quiz evidence
- Implement the baseline schedule generator (topic/subtopic dependency order + difficulty + synthetic estimates → static per-subject sequence, merged across a student's enrolled subjects)
- ****Build deadline-aware prioritization****: for each schedulable unit, its urgency deadline is whichever exam date applies to its module (UT1 date if `assessment\_phase` = UT1`, UT2 date otherwise), with end-sem as the final backstop deadline for everything — items closer to their deadline and lower confidence get pulled forward
- Implement the adaptive re-scheduling function (given availability + current confidence ratings + remaining units + deadlines → updated day-by-day plan)

- Validate all logic against Member A's synthetic student profiles before real integration

****Deliverable:**** a self-contained scheduling/mastery module (pure Python, testable independent of the web layer) that B can hand off as a library/service.

Member C — Application & Interface

- Backend API (e.g. FastAPI) with endpoints: enrollment (elective + honors + availability), get merged schedule across all subjects, submit confidence rating, get updated schedule
- Integrate Supabase Auth for student login/signup
- Frontend: enrollment flow (core auto-shown, elective picker, honors picker), calendar/schedule view, availability form, confidence check-in UI
- Wire the backend to Supabase and to Member B's engine module

****Deliverable:**** a working end-to-end app that a student can actually sign up on, enroll into their subjects, and use.

Sequencing

- ****Week 1:**** all three align on the schema together (30–60 min), since A's schema decisions affect B's data access and C's API contracts. After that, work is parallel.
- ****Mid-project checkpoint:**** B integrates against A's real seed data (not just synthetic) once it's ready; C integrates B's engine behind the API.
- ****Final integration:**** full walkthrough — onboard a fake student, run through a week of simulated check-ins, confirm the schedule adapts as expected.

6. Open Questions

- Any hard demo/deadline date that should shape which phase (baseline-only vs. full adaptive loop) is the "must-have" cutoff for the team?
- For each subject, is the UT1/UT2 "half syllabus" split explicitly defined by the teacher (which modules go where), or does Member A need to infer/propose a split for confirmation?

7. Future Work (explicitly not in this MVP)

- Mentor chat via a locally-hosted LLM (e.g. Ollama + a small model) with RAG grounded in the syllabus content
- Generalizing back to multi-syllabus / multi-course support with real document parsing